

SubEco · 구독 관리 트래커

기술 설계 보고서

Spring Boot 4 · JPA · Mustache · REST API 기반
구독 관리 시스템의 기획 · 아키텍처 · 구현 설계

평가 범위 스프링부트 1강 ~ 14강 (MVC · JPA · CRUD · REST API · JSON)

기술 스택 Spring Boot 4.0.6 · Java 17 · Spring Data JPA · H2 · Mustache · RestClient

팀 구성 5명 — 이태경(기획·도메인) · 김세진(CRUD·REST) · 이상윤(환율·테스트) · 김준아(뷰 데이터) · 이예은(UI·UX)

작성일 2026년 6월

01 서비스 기획 — 아이디어 · 비즈니스

1.1 서비스 소개

SubEco — 흩어진 구독 서비스의 결제일·금액·통화를 한 곳에서 관리해 주는 **가계부형 구독 대시보드**입니다.

- **타겟 사용자** — Netflix·Spotify·ChatGPT 등 여러 구독을 동시에 쓰는 2030 세대. "지금 몇 개를, 얼마에, 언제 결제 중인지" 파악하지 못하는 사람.
- **핵심 기능** — 구독 등록/조회/수정/삭제, 결제주기 기반 **다음 갱신일 자동 계산**, 외화 구독의 **실시간 환율 원화 환산**, 갱신 임박 알림, 월 합산 지출 대시보드.

1.2 기획 배경 — 왜 필요한가, 수업 예제와 무엇이 다른가

구독 경제가 커지면서 한 사람이 동시에 5~10개의 구독을 쓰는 일이 흔해졌습니다. 그러나 결제일이 제각각이라 "이번 달에 얼마가 빠져나가는지", "안 쓰는 구독이 자동결제되고 있지는 않은지"를 한눈에 보기 어렵습니다. SubEco는 이 문제를 해결합니다.

구분	수업 예제 (Article 게시판)	SubEco
데이터 성격	정적인 글(제목·본문)	날짜·금액·통화를 가진 계산 대상
핵심 로직	단순 저장·조회	갱신일 자동 계산 · 환율 환산 · 임박 판별
부가 가치	없음(CRUD 자체가 목적)	"언제·얼마" 인사이트 제공

1.3 사용 시나리오

1. 사용자가 Netflix·ChatGPT Plus 등 자신의 구독을 **등록**한다(서비스명·금액·통화·주기·시작일·태그).
2. 홈 대시보드에서 **월 합산 지출(원화 환산)**과 **7일 이내 갱신 예정 목록**을 확인한다.
3. 외화 구독(USD/JPY)은 **실시간 환율로 환산된 원화**가 함께 표시된다.
4. 갱신이 임박한 구독은 카드에 **강조 배지**로 알려 준다.
5. 더 이상 쓰지 않는 구독은 **삭제**해 고정비를 정리한다.

1.4 비즈니스 가능성

- **수익 모델** — 갱신 임박 시점에 더 저렴한 요금제·대체 서비스를 안내하는 **제휴/광고 추천**, 안 쓰는 구독 정리를 돕는 **구독 다이어트 코칭(프리미엄)**.
- **확장 방향** — 사용자 인증 연동(개인별 구독함), 카드 명세서 연동 자동 등록, 가족 공유 구독 분담 계산.
- **사회적 가치** — 불필요한 자동결제로 인한 **고정비 누수를 줄여** 합리적 소비를 돕는다.

1.5 사용자 경험(UX) 배려

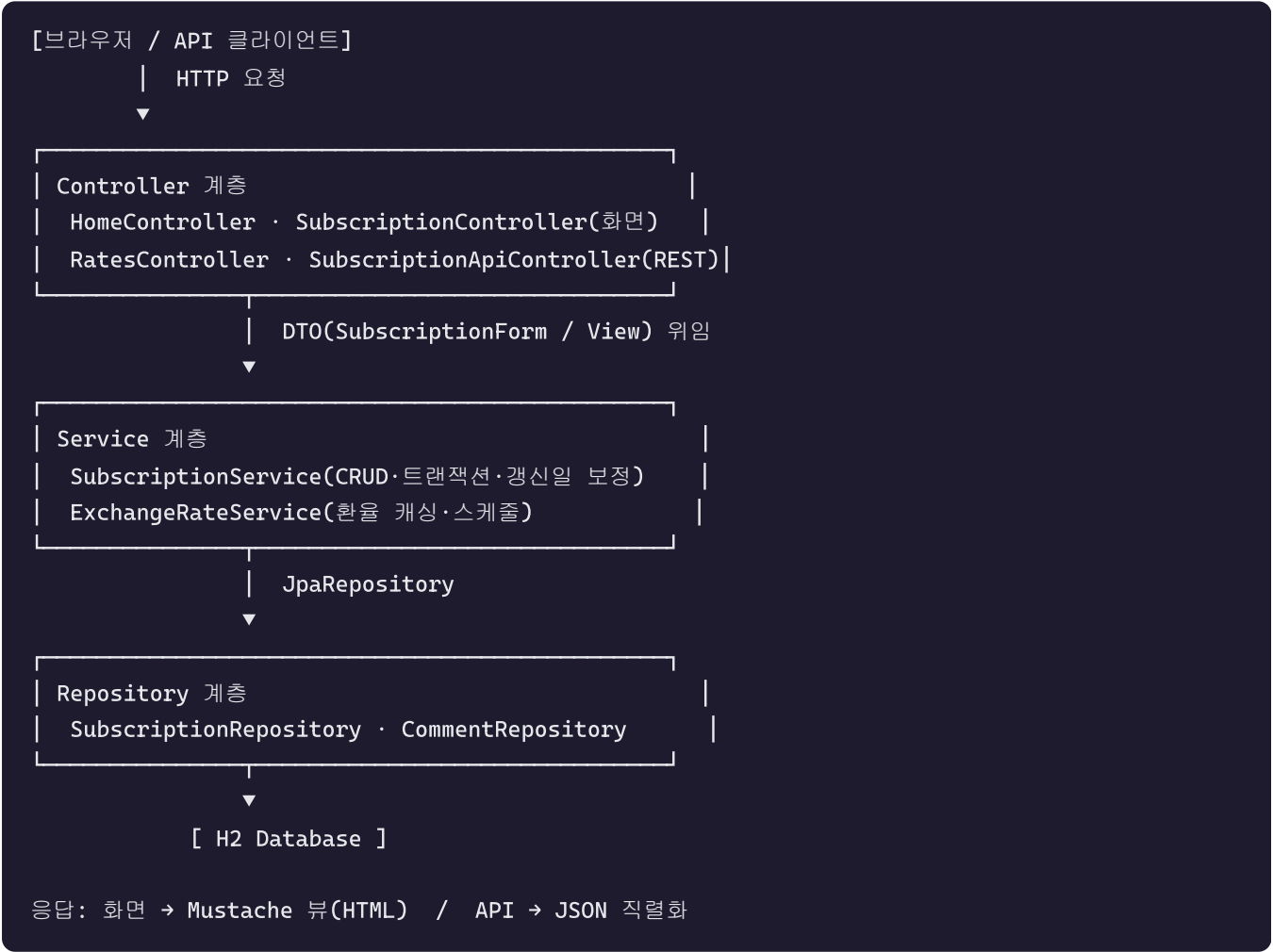
UX 요소	구현 위치
구독이 없을 때 빈 목록 안내 문구	Mustache 조건 섹션
없는 구독·잘못된 주소 시 커스텀 404 안내 페이지	<code>GlobalExceptionHandler</code> → <code>error/404</code>
갱신 임박(3·7일) 강조 배지	<code>SubscriptionView.dueSoon</code> / <code>dueSoon7</code>
외화 금액에 원화 환산 병기	<code>SubscriptionView.displayPrice</code>
환율 API 실패 시 실시간/폴백 상태 표시 + 수동 갱신	<code>/rates</code> 페이지

02 기술 스택 & 아키텍처

2.1 기술 스택

구분	기술	비고
백엔드	Spring Boot 4.0.6 · Java 17	Gradle 빌드
웹 / 뷰	Spring MVC · Mustache(SSR)	로직 없는 템플릿 엔진
ORM	Spring Data JPA / Hibernate	Repository 자동 구현
DB	H2 (In-Memory)	<code>data.sql</code> 시드
HTTP / JSON	RestClient · Jackson 3	외부 환율 API 연동
테스트	JUnit 5 · MockMvc	REST 통합 · 리파지터리
기타	Lombok	보일러플레이트 제거

2.2 계층 흐름 (팀 코드 기준)



각 계층은 자신의 책임에만 집중하고, 의존 방향은 **Controller → Service → Repository** 단방향으로만 흐른다. 화면용/REST용 컨트롤러를 분리해 같은 비즈니스 로직(`SubscriptionService`)을 재사용한다.

2.3 패키지 구성

```
com.framework.subscriptions
├─ controller    Home · Subscription · SubscriptionApi · Rates · GlobalExceptionHandler
├─ service       SubscriptionService · ExchangeRateService
├─ repository    SubscriptionRepository · CommentRepository
├─ domain        Subscription · Comment · BillingCycle · Currency · Tag ·
SubscriptionNotFoundException
└─ dto           SubscriptionForm(입력) · SubscriptionView(출력)
```

2.4 주요 어노테이션 역할

어노테이션	역할
<code>@Entity</code> · <code>@Id</code> · <code>@GeneratedValue</code>	JPA 엔티티와 기본키 지정(H2 IDENTITY 자동 증가)
<code>@Enumerated(EnumType.STRING)</code>	enum을 문자열로 저장 — ORDINAL 순서 변경 위험 회피
<code>@ElementCollection</code> · <code>@CollectionTable</code>	태그 집합을 별도 조인 테이블(<code>subscription_tags</code>)에 값 타입으로 저장
<code>@ManyToOne</code> · <code>@JoinColumn</code>	댓글→구독 단방향 다대일 연관, FK 컬럼 지정
<code>@Builder</code> · <code>@Getter</code> · <code>@NoArgsConstructor(PROTECTED)</code>	Lombok — 빌더 생성만 허용해 엔티티 생성 캡슐화
<code>@Service</code> · <code>@RequiredArgsConstructor</code>	서비스 빈 등록 + final 필드 생성자 주입
<code>@Transactional(readOnly = true)</code>	트랜잭션 경계 설정, 기본 읽기 전용(변경 메서드만 재정의)
<code>@Controller</code> / <code>@RestController</code>	뷰 이름 반환 / 반환 객체를 JSON 본문으로 직렬화(= <code>@Controller</code> + <code>@ResponseBody</code>)
<code>@GetMapping</code> · <code>@PostMapping</code> · <code>@PatchMapping</code> · <code>@DeleteMapping</code>	HTTP 메서드별 URL 매핑
<code>@PathVariable</code> · <code>@RequestParam</code> · <code>@RequestBody</code>	경로변수 · 쿼리파라미터 · JSON 본문 바인딩
<code>@ControllerAdvice</code> · <code>@ExceptionHandler</code> · <code>@ResponseStatus</code>	전역/지역 예외 처리와 응답 상태코드 매핑
<code>@PostConstruct</code> · <code>@Scheduled</code>	빈 초기화 직후 1회 실행 / cron 주기 실행(환율 갱신)
<code>@SpringBootTest</code> · <code>@AutoConfigureMockMvc</code>	전체 컨텍스트 통합 테스트 / MockMvc 자동 구성

03 도메인 모델 설계

3.1 Subscription 엔티티

구독 1건을 표현하는 핵심 엔티티. setter를 열지 않고 빌더 생성 + 의도가 드러나는 메서드 (`updateDetails`, `slideToNextCycle`)로만 상태를 바꾼다.

```
// 결제주기 기반 다음 갱신일 = 시작일 + 주기
public LocalDate getNextRenewalDate() {
    return billingCycle.addTo(startedAt);
}
```

3.2 enum에 로직 내장 — BillingCycle / Currency / Tag

- `BillingCycle` (MONTHLY/YEARLY) — `addTo(LocalDate)`로 날짜 계산 로직을 enum 안에 캡슐화 (switch 표현식).
- `Currency` (KRW/USD/JPY) — 표시 라벨 + 풀백 환율 보유. 환율 API 실패 시 사용.
- `Tag` (ENTERTAINMENT/MUSIC/AI/PRODUCTIVITY/GAME/DEV) — 분류용. `@ElementCollection`으로 저장.

3.3 Comment 엔티티 (14강)

구독에 메모(댓글)를 남기는 도메인. 구독 → 댓글 단방향 @ManyToOne만 두어 엔티티를 단순하게 유지하고, 조회는 리파지터리 메서드 쿼리로 해결한다.

```
@ManyToOne
@JoinColumn(name = "subscription_id", nullable = false)
private Subscription subscription; // nickname · body
```

3.4 DTO — 입력/출력 분리

DTO	방향	역할
<code>SubscriptionForm</code>	입력	폼/JSON 입력 바인딩 → <code>toEntity()</code> 로 엔티티 변환
<code>SubscriptionView</code>	출력	화면·JSON에 필요한 모든 값을 생성자에서 미리 계산한 불변 객체

Mustache는 로직을 쓸 수 없으므로 `displayPrice`, `daysText`, `dueSoon`, `colorClass`, `emoji`, `monthlyKrw` 등 화면 계산을 모두 `SubscriptionView` 생성자에서 끝낸다. 템플릿은 출력만 담당.

04 CRUD 기능 설계

4.1 SubscriptionService — 트랜잭션 전략

```
@Service
@Transactional(readOnly = true) // 클래스 기본: 읽기 전용
public class SubscriptionService {
    @Transactional // 변경 메서드만 쓰기 트랜잭션
    public Subscription create(SubscriptionForm form) { ... }
}
```

읽기 전용 트랜잭션은 성능 이점과 함께 조회 메서드에서의 실수 변경을 예방한다. 단, `findAll` / `findById` 는 조회 시 갱신일을 보정·저장하므로 쓰기 트랜잭션으로 둔다.

4.2 갱신일 자동 보정

```
// 지난 갱신일을 오늘 이후가 될 때까지 주기를 밀어준다
while (!subscription.getNextRenewalDate().isAfter(today)) {
    subscription.slideToNextCycle();
}
```

별도 스케줄러 없이 조회 시점에 실시간 보정해, 며칠을 방치해도 항상 정확한 미래 갱신일을 반환한다.

4.3 더티 체크 기반 update

`@Transactional` 범위에서 영속 엔티티의 필드를 바꾸면 JPA가 변경을 감지해 커밋 시 자동 UPDATE를 실행한다. `save()` 호출이 없어 코드가 간결하다.

4.4 메서드 요약

메서드	역할	특이사항
<code>findAll(tag)</code>	전체/태그 조회	갱신일 보정 + 태그 필터
<code>findById(id)</code>	단건 조회	없으면 404 예외
<code>create(form)</code>	신규 저장	Form → Entity 변환
<code>update(id, form)</code>	수정	더티 체크(save 없음)
<code>delete(id)</code>	삭제	존재 확인 후 삭제, 없으면 404

4.5 화면 컨트롤러 · PRG 패턴

`SubscriptionController` 는 `/subscriptions` 아래 7개 엔드포인트(목록·등록폼·저장·상세·수정폼·수정·삭제)를 제공한다. HTML form은 GET/POST만 지원하므로 수정·삭제도 POST로 받고, 처리 후 **PRG 패턴**으로 `redirect:` 시켜 새로고침 중복 제출을 막는다. 입력은 엔티티가 아닌 `SubscriptionForm` DTO로 받아 엔티티 노출을 막는다.

05 REST API / JSON 설계

`@RestController` 인 `SubscriptionApiController`가 `/api/subscriptions` 경로로 JSON CRUD를 제공한다. 반환 객체는 Jackson이 자동으로 JSON 직렬화하며, 읽기 응답은 화면과 동일한 `SubscriptionView`를 재사용한다.

5.1 엔드포인트

메서드	경로	응답
GET	<code>/api/subscriptions</code>	200 · 구독 목록 JSON 배열
GET	<code>/api/subscriptions/{id}</code>	200 · 단건 JSON (없으면 404 JSON)
POST	<code>/api/subscriptions</code>	201 · 생성된 구독 JSON
PATCH	<code>/api/subscriptions/{id}</code>	200 · 수정된 구독 JSON
DELETE	<code>/api/subscriptions/{id}</code>	204 · 본문 없음

5.2 응답 예시

```
// GET /api/subscriptions/1
{"id":1,"serviceName":"Netflix","price":17000,"currency":"KRW",
 "billingCycle":"MONTHLY","displayPrice":"17,000 원","daysText":"27일 후",
 "dueSoon":false,"tags":["ENTERTAINMENT"], ... }

// GET /api/subscriptions/9999 → 404
{"message":"구독 정보를 찾을 수 없습니다. (id=9999)"}

```

5.3 REST 전용 JSON 404 — 컨트롤러 로컬 예외 핸들러

전역 `GlobalExceptionHandler`는 화면용 `error/404` HTML 뷰를 반환한다. 이를 REST에 그대로 적용하면 JSON 대신 HTML이 나가므로, REST 컨트롤러 안에 **컨트롤러 로컬** `@ExceptionHandler`를 두어 같은 예외를 JSON 404로 응답한다(지역 핸들러가 전역보다 우선).

검증: `curl`로 목록·단건·생성(201)·수정(200)·삭제(204)·404(JSON) 전 케이스 확인 완료. **제출 시 브라우저/Postman 응답 캡처를 첨부할 것.**

06 환율 연동 설계

ExchangeRateService가 외화 구독을 원화로 환산한다.

- `@PostConstruct`로 앱 시작 시 1회 호출 → `ConcurrentHashMap` 캐싱, `@Scheduled(cron="0 0 0 * * *")`로 매일 자정 갱신.
- API 실패 시 `try/catch`로 `Currency` enum 풀백 환율 유지 → 앱 무중단.
- JPY는 직접 환율 대신 USD 기준 교차 환율($KRW/USD \div JPY/USD$)로 계산.
- `RestClient`로 호출하고 응답을 `Map<String, Object>`로 받아 역직렬화 문제를 회피.

07 테스트 설계

총 9개 테스트 전부 통과.

테스트	개수	검증
<code>SubscriptionApiControllerTest</code> (MockMvc)	6	목록·단건·404·생성·수정·삭제 전 시나리오
<code>CommentRepositoryTest</code>	2	<code>findBySubscriptionId</code> 시드 댓글 조회
<code>SubscriptionsApplicationTests</code>	1	컨텍스트 로드

```
mockMvc.perform(post("/api/subscriptions") ... )
    .andExpect(status().isCreated())
    .andExpect(jsonPath("$.serviceName").value("TestSub"));
```

08 발생한 문제 & 해결 과정

① 환율 API가 KRW 미지원 — JSON 대신 HTML 에러 반환 외부 API 네트워크

현상 Frankfurter API 호출 시 `text/html` 에러 페이지가 와 파싱 실패. 핫스팟으로 바뀌도 동일.

원인 네트워크가 아니라 데이터 소스(ECB)에 원화(KRW)가 없어서였음.

해결 KRW 포함 170개 통화를 지원하는 `open.er-api.com` 으로 교체.

② Jackson record 역직렬화 / unknown property 위험 Jackson 설계

현상 커스텀 `record` 로 응답 매핑 시 생성자 접근·미지정 필드 오류 가능성.

해결 응답을 `Map<String, Object>` + `ParameterizedTypeReference` 로 받아 unknown property 개념 자체를 제거. 커스텀 record-Jackson 직접 import 불필요.

③ Spring Boot 4에서 RestClient.Builder 빈 미등록 → 기동 실패 런타임 Spring Boot 4

현상 생성자에 `RestClient.Builder` 주입 시 `UnsatisfiedDependencyException` 으로 서버가 뜨지 않음.

해결 빈 주입 대신 `RestClient.create()` 로 직접 생성 — 자동 구성 의존을 제거.

④ REST 테스트 작성 시 import 패키지가 달라짐 (Jackson 3 · 모듈형 스타터) 컴파일 Spring Boot 4

현상 `com.fasterxml.jackson.databind`, `... autoconfigure.web.servlet` 패키지를 찾을 수 없다는 컴파일 오류 4건.

원인 Boot 4는 Jackson 3(`tools.jackson`)과 모듈형 스타터를 채택 → `@AutoConfigureMockMvc` 가 `org.springframework.boot.webmvc.test.autoconfigure` 로 이동.

해결 추측 대신 클래스패스 jar 내부를 직접 확인해 실제 패키지로 import 교정 → 테스트 9개 전부 통과.

09 결론 & 개선 방향

9.1 설계 결정 요약

설계 결정	이유
<code>readOnly=true</code> 기본 트랜잭션	성능 + 조회 메서드 실수 변경 방지
갱신일 자동 보정	스케줄러 없이 항상 정확한 갱신일
더티 체크 update	불필요한 <code>save()</code> 제거
Form/View DTO 분리	엔티티 불변성 · 관심사 분리 · 뷰 계산 격리
화면/REST 컨트롤러 분리	같은 서비스 재사용, 응답 형식만 분리(JSON 404 포함)
댓글 단방향 <code>@ManyToOne</code>	엔티티 단순화, 메서드 쿼리로 조회
enum에 로직/폴백 내장	날짜·환율 규칙을 한곳에 응집

9.2 향후 개선 방향

- 사용자 인증(Security) — 구독을 계정에 연결해 개인화 목록 제공.
- 댓글 기능 완성 — 현재 엔티티·리파지터리까지(14강). 컨트롤러·서비스·UI로 확장(15강 이후).
- 알림 — 갱신 N일 전 이메일/푸시(Spring Scheduler 활용).
- 페이지네이션 — 구독이 많아지면 `Pageable` 기반 페이징과 쿼리 단계 집계.

9.3 마무리

SubEco는 1~14강의 MVC·JPA·CRUD·REST API·JSON을 실제 구독 관리라는 주제로 구현하면서, 갱신일 자동 계산·실시간 환율·대시보드로 수업 예제와 차별화했다. 모든 계층이 책임을 분리해, 한 부분이 바뀌어도 영향이 최소화되는 구조를 갖췄다.